

从函数回调到 Agent Loop 调试

学习笔记 · 回调的本质 · 闭包捕获时机 · 错误恢复路径排错

目录

- 1. 回调是思想还是语言特性
- 2. 怎么判断一个函数该被回调还是直接执行
- 3. 核心机制：闭包的"捕获时机"
- 4. 实战：Agent Loop 里的模型 fallback 失效 Bug
- 5. 三层模型：能不能切模型取决于什么
- 6. join 的陷阱：到底在连接谁
- 7. 错误恢复路径自身的脆弱性
- 8. 速查表与要点回顾

1. 回调是思想还是语言特性

结论：回调本质是一种**思想**（设计模式 / 编程范式），不是某个具体语言特性。但它写得顺不顺，取决于语言提供了哪些特性。

回调描述的是一种**控制关系**：你把一段可执行逻辑交给另一个函数，由后者决定在*什么时候*回头调用它。这个"由别人决定何时调用我"就是**控制反转**（inversion of control）——正常调用是你调库，回调是库反过来调你。

判断它是思想而非特性，最直接的证据：几乎每种语言都有回调，但实现机制天差地别。同一个模式，不同机制：

- C 用函数指针
- C++ 用函数指针 / 仿函数 / lambda / `std::function`
- Java 8 之前没有一等函数，只能用接口 + 匿名内部类拿对象冒充
- JS / Python / Rust / Kotlin 用闭包
- C# 用委托

需要区分层次：回调本身是思想，但**一等函数**（first-class function）和**闭包**（closure）确实是语言特性——它们是让回调这个思想表达得最自然的那批特性。

底层统一视角

几乎所有回调机制剥到底都是同一件事：**一次目标地址在运行期才确定的间接调用**（indirect call），外加可选的它携带的状态。普通调用是 `call <固定地址>`（编译期写死），回调是 `call <寄存器/内存>`（地址运行期才算出）。CPU 执行的是同一条 `call` 指令，区别只在目标地址是写死还是算出来的。

能力递增的实现路径：

- **函数指针**：只指向代码，不能带状态。所以 C 的回调 API 几乎都额外带 `void* user_data` 手动传上下文——这其实是在手动模拟"闭包捕获的环境"。
- **闭包** $\approx$ (代码指针, 环境)。环境就是从外层作用域捕获的变量。这也是为什么闭包比函数指针"胖"。
- `std::function`：类型擦除，能装任意可调用物，代价是一次间接调用 + 可能堆分配。
- **虚函数 / 接口**：用对象做回调，底层走 vtable 虚分发。**深层联系**：回调和多态其实是同一回事——都是调用方编译期不知道执行哪段代码，运行期通过一层间接才确定。

2. 怎么判断一个函数该被回调还是直接执行

核心认识：一个函数该被回调还是直接执行，**不是它自身的属性，而是它和调用方那层关系的属性**。同一段逻辑，换个上下文就能从"直接调"变成"被回调"。别盯函数本身，盯那次调用。

判断只需问一个问题：在"该执行它"的那一刻，是**我**（写调用代码的人）知道何时调，还是**另一边**知道？我知道、就是现在、顺着往下走 → 直接调。只有另一边知道"什么时候 / 要不要 / 调几次" → 交出去让另一边回调。

需要回调的两类根本原因

1. **时机或次数不归你管**。执行的时刻还没到，或会来很多次。I/O 完成、定时器触发、网络返回、按钮被点——本质是**时间轴上的错位**：执行点和你写代码的点不重合。
2. **驱动逻辑的代码不是你写的**。真正跑循环 / 控主流程的是通用的框架代码，它需要在某个点插进你的特定逻辑。`sort` 拥有排序算法，你只拥有"两个元素怎么比"。深层约束是**依赖方向**：通用代码不能反过来依赖你的具体代码，所以你的逻辑只能作为参数注入。

两个实用判断手法

① **返回值替代测试**。能不能改成"调用函数、拿返回值用"而不损失任何东西？能 → 不需要回调，`int s = add(a, b)` 就够了。回调争到位置的前提，是普通返回值表达不了这层关系：结果**稍后**才来（异步）、会来多次（流/事件）、或**不是你驱动**（框架循环）。

② **签名是谁定的**。写普通函数时参数由你定；写回调时框架早就决定了甩给你什么。一旦你发现自己在写"参数是框架给我什么我就收什么"的函数，那它就是回调。并行特征：**回调你注册出去，但从不自自己调它**，调用权在别人手里。

放回语境：**hook 是回调的典型**。你写的脚本签名（能拿到什么上下文、该返回什么）是 harness 规定的，你注册到 `PreToolUse` 之类的点上，从不自己调用，由 harness 在生命周期节点回头调。命中第二类：驱动流程的是 harness，不是你。

3. 核心机制：闭包的"捕获时机"

这是贯穿整次调试的关键。Python 里有一个极易踩的点：**lambda / 函数的默认参数 = 定义时按值冻结**。默认值只在函数对象创建那一刻求值一次。

```
# 默认参数在 lambda 定义时立即求值一次，从此冻死
lambda mt=max_tokens, mdl=state.current_model: ...

# 对比：自由变量在每次调用时才查
lambda: client.create(model=state.current_model) # 调用时才读 state
```

一句话：用默认参数捕获 = 冻结在**定义时**；用自由变量捕获 = 读取在**调用时**。这个差别在"对象会被中途修改、且函数被反复调用"的场景下是实打实的 Bug 来源。

写法	捕获时机	适用场景
<code>def f(x=val) / lambda 默认参数</code>	定义时冻结（按值）	值在重试期间 不变 ，想锁住快照
读自由变量 / 具名 def 读外部	调用时读取（实时）	值会被中途改、要读到 最新

4. 实战：Agent Loop 里的模型 fallback 失效 Bug

这是教学项目 `s11` 参考代码里一个隐性 Bug，也是整次排查的核心案例。

现象

日志会打印 `switching to <fallback>`，但实际发出的请求 `model` 字段**纹丝不动**，永远是主模型。"日志说切了、请求没切"——这种 Bug 最坑，现象会骗你。

带 Bug 的代码

```
response = with_retry(
    lambda mt=max_tokens, mdl=state.current_model: # mdl 在定义时冻死
        client.messages.create(model=mdl, ...),
    state)
```

根因（时间轴拆解）

- lambda **定义时**，默认参数 `mdl=state.current_model` 立刻求值，锁成 `PRIMARY_MODEL`。
- `with_retry` 反复调的是**同一个** lambda 对象，默认参数不会因重新调用而重算。
- 第 N 次 529 时，`with_retry` 尽职地把 `state.current_model` 改成了 `FALLBACK_MODEL`。
- 但下一次 `fn()` 里 `model=mdl` 用的还是冻死的 `PRIMARY_MODEL`。**state 被改了，但 callback 根本不读它**，它读的是冻死的 mdl。

更糟的是这个 lambda 在 `while True` 每轮迭代重建，理论上一轮能"追上"——但 `with_retry` 在切换后仍于**当轮** retry 循环内用旧模型继续重试，直至 `MAX_RETRIES` 耗尽抛 `RuntimeError`，被外层判 unrecoverable 退出。**在最该靠 fallback 救命的持续过载场景下，loop 往往在用上新模型之前就已退出**。

修复

```
# 方案 A：具名 def, model 调用时读取（推荐）
def call_model():
    return client.messages.create(
        model=state.current_model, # 调用时读取, fallback 才生效
        system=system, messages=messages,
        tools=T00LS, max_tokens=max_tokens)

response = with_retry(call_model, state)

# 方案 B：保留 lambda, 只删 mdl 默认参数
response = with_retry(
    lambda mt=max_tokens: # 只冻 max_tokens
        client.messages.create(
            model=state.current_model, # 关键：改为调用时读 state
            ...),
    state)
```

`max_tokens` 在重试期间不被修改，保留 `mt=max_tokens` 冻结无影响；但 `model` 必须解冻。一个侥幸正确、一个致命错误，差别只在"被冻的值在重试期间会不会变、由谁改"。

5. 三层模型：能不能切模型取决于什么

追了好几轮才收敛出来的结论：**能不能切模型，从来不在 `PRIMARY_MODEL`，也不在 `RecoveryState`，而在 `callback` 读 `model` 的时机**。三层必须同时成立：

层	关注点	判断
数据层	<code>current_model</code> 是不是可变字段	是普通实例属性，可改 ✓
逻辑层	<code>with_retry</code> 有没有真的写 <code>state.current_model = ...</code>	529 分支确实改了 ✓
捕获层	<code>callback</code> 读 <code>model</code> 是定义时冻 还是 调用时读	默认参数冻死 → ✗ 断点在此

几个关键澄清：

- `PRIMARY_MODEL = os.environ["MODEL_ID"]` 读的是环境变量，永远是一个**字符串**。即便写成逗号分隔的多模型，Python 拿到的也只是带逗号的普通字符串，除非显式 `.split(",")`，否则在程序眼里是单个值。
- `self.current_model = PRIMARY_MODEL` 只是**拷贝当前值**。从这一刻起两者再无瓜葛，改 `self.current_model` 不会动 `PRIMARY_MODEL`。
- 字段可写 ≠ 请求一定读到新值**。这正是为什么要分三层——数据层和逻辑层都对了，捕获层一处冻结就把下游所有努力废掉。

6. join 的陷阱：到底在连接谁

关键区别：`join` 的输入是不是同一个东西。`str.join(x)` 的语义是遍历 `x`、把每个元素连起来。结果取决于"`x` 的元素是什么"。而**字符串本身就是可迭代的，它的元素是单个字符**——这是钥匙。

写法	括号里是什么	结果
<code>" ".join("hello")</code>	一个 字符串	<code>"h e l l o"</code> ✗ 拆成字母
<code>" ".join(["hello","world"])</code>	一个 列表	<code>"hello world"</code> ✓

错误写法及其两个毛病

```
# ✗ 错：join 收到的是单个字符串 str(...)，被拆成字母
# ✗ 错：在 for 循环里每轮覆盖 content，多个 block 只剩最后一个
if getattr(b, "type", None) == "text":
    content = " ".join(str(getattr(b, "text", "")))
```

正确写法

```
# ✓ generator 版：join 收到的是逐个吐出字符串的生成器
content = " ".join(
    str(getattr(b, "text", "")) for b in content
    if getattr(b, "type", None) == "text"
)

# ✓ list 版：先收集成列表再 join，结果相同
texts = [str(getattr(b, "text", "")) for b in content
          if getattr(b, "type", None) == "text"]
content = " ".join(texts)
```

两个版本结果完全一样。generator 版不建中间列表、边产边连，更省内存，是 `join` 场景更地道的写法（`join` 接受任意可迭代对象）。这种写法同时解决了"拆成字母"和"只剩最后一个"两个问题——天然把所有 `text block` 一起收进来连接。

7. 错误恢复路径自身的脆弱性

这次还暴露了两类恢复逻辑的毛病——共性是"**搭了对的一半，但接错了线**"。

① 守卫接错变量（软循环）

想做"reactive compact 只重试几次"，定义了 `MAX_REACTIVE_RETRIES` 和 `reactive_retries` 计数器，**却把守卫判断接到了另一个字段**（`state.has_attempted_reactive_compact`，且从没赋 `True`）。结果计数器空转，`return` 不可达 → 可能无限循环 + 烧 token。

```
# 修复 A：守卫改成读你真正维护的计数器
if reactive_retries < MAX_REACTIVE_RETRIES:
    message[:] = reactive_compact(message)
    reactive_retries += 1          # 这下它真的有用
    continue

# 修复 B：用布尔开关，但要记得真的把它翻成 True
if not state.has_attempted_reactive_compact:
```

```
message[:] = reactive_compact(message)
state.has_attempted_reactive_compact = True    # 别漏这句
continue
```

两套机制别混用，留一套删一套。注意语义差别：局部变量 `reactive_retries` 每次进函数重置；`state` 上的字段跨整个会话累计。

② 恢复路径里的裸 API 调用（硬崩）

`summarize_history` 里的 `client.messages.create` 没有 `try/except`、也没包 `with_retry`。它在 `except` 块体内被调用，而 `except` 块体自身新抛的异常不被同一个 `except` 捕获，会逃出 `agent_loop`、再逃出 `__main__`（也没兜），整个进程带 `traceback` 崩溃。

讽刺点：正是在"主调用已因 `prompt_too_long` 失败、最需要稳妥"的当口，恢复代码自己又发了一个不设防的 API 请求。**恢复逻辑比主逻辑更脆。**

```
def summarize_history(messages):
    ...
    try:
        response = client.messages.create(model=MODEL, ...)
    except Exception as e:
        print(f"[summarize failed] {type(e).__name__}: {e}")
        return "(summary unavailable - summarization call failed)"    # 降级，不抛
    return ...
```

两者要配合看：裸调用导致"硬崩"，守卫失效导致"软循环"。降级摘要后 `prompt` 可能仍超长，又会触发 `prompt_too_long`——这时全靠守卫兜底，让它试够次数后干净走 `[unrecoverable]` 退出。两个都修好，恢复路径才算稳。

附：`retry_delay` 的"预留口子没接线"

`retry_delay(attempt, retry_after=None)` 整体在用、逻辑正确（指数退避 + jitter + 32s 封顶），但 `retry_after` 形参从没被传过实参，两处都是 `retry_delay(attempt)`。于是"优先用服务器 `Retry-After`"那条分支永远不进——接口备好了，没接线，和守卫那个毛病同类。要生效需在 `with_retry` 捕获异常时从 SDK 异常的响应头取 `retry-after` 传进去。

8. 速查表与要点回顾

主题	一句话要点
回调的性质	是思想（控制反转），不是语言特性；一等函数 / 闭包才是特性
底层统一	回调 = 运行期才定地址的间接 call + 可选状态；与多态同源
是否该回调	看"该执行那一刻谁知道时机"；返回值能替代就别回调
认出回调	签名由框架定 + 你注册但从不自己调它
默认参数捕获	定义时按值冻结；值会变就别用它，改自由变量调用时读
fallback 失效	数据层/逻辑层对了，捕获层一冻就全废；三层须同时成立
环境变量	永远是字符串，多模型需显式 <code>.split(",")</code>
join	看括号里喂的是 <code>str</code> 还是可迭代容器； <code>str</code> 会被拆成字母
恢复路径	守卫接对变量（防软循环）+ 裸 API 调用要兜异常（防硬崩）
预留参数	形参留了口子不等于接了线，确认上游真的传值

贯穿主线：这次几乎所有 Bug 都是同一类毛病——"**搭了对的一半，但接错了线**"。fallback 把数据/逻辑都做对了，却被默认参数在捕获层冻死；`reactive_retries` 计数器建好了却没接到守卫上；`retry_after` 形参留了却没传值。修的不是"补功能"，而是"把已有的两半接通"。